

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 7 · POO avanzada: enum, wrappers, StringBuilder, equals y hashCode

Tipos enumerados (enum): valores, ordinal(), name() y valueOf()

Clases envoltorio (wrappers): autoboxing, unboxing y caché de Integer

String vs StringBuilder vs StringBuffer: inmutabilidad y eficiencia

El contrato equals() y hashCode()

Composición y agregación entre clases. Objetos inmutables

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 2
Valoración	POO avanzada y clases de utilidad

Parte 1: Opción múltiple y Verdadero/Falso

4 puntos — 0,5 ca

Dado el enum: `enum Dia { LUNES, MARTES, MIÉRCOLES, JUEVES }`, ¿qué devuelve `Dia.MARTES.ordinal()`?

- a) 0
- b) 1
- c) 2
- d) Error de compilación

RESPUESTA / SOLUCIÓN

b) 1 — `ordinal()` devuelve la posición del valor dentro del enum empezando en 0 (`LUNES=0`, `MARTES=1`).

¿Qué imprime este código?

```
Integer a = 500;
Integer b = 500;
System.out.println(a == b);
```

- a) true
- b) false
- c) Error de compilación
- d) `NullPointerException`

RESPUESTA / SOLUCIÓN

b) false — `a == b` compara referencias, y el valor 500 está fuera de la caché de Integer (-128..127), por lo que son objetos distintos. Para comparar valores debe usarse `a.equals(b)` o `a.intValue() == b.intValue()`.

StringBuilder es una clase inmutable: una vez creado un objeto StringBuilder no se puede modificar su contenido.

RESPUESTA / SOLUCIÓN

Falso — `StringBuilder` es mutable: métodos como `append()`, `insert()` o `reverse()` modifican el mismo objeto. La clase inmutable es `String`.

Con estas declaraciones, ¿qué línea imprime true?

```
String s1 = new String("hola");
String s2 = new String("hola");
System.out.println(s1 == s2);          // línea 1
System.out.println(s1.equals(s2));     // línea 2
```

- a) Solo la línea 1
- b) Solo la línea 2
- c) Ambas líneas
- d) Ninguna

RESPUESTA / SOLUCIÓN

b) Solo la línea 2 — `==` compara referencias (`s1` y `s2` son objetos distintos → false), mientras que `equals()` compara el contenido («hola» == «hola» → true).

¿Qué imprime este código?

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b);
```

- a) true
- b) false
- c) Error de compilación
- d) 100

RESPUESTA / SOLUCIÓN

a) true — Los valores entre -128 y 127 están en la caché de Integer, por lo que a y b referencian el mismo objeto y == devuelve true.

Si una clase sobrescribe equals(), debe sobrescribir también hashCode() para cumplir el contrato: dos objetos iguales según equals() deben tener el mismo hashCode().

RESPUESTA / SOLUCIÓN

Verdadero — Si no se cumple, estructuras como HashMap o HashSet pueden fallar (objetos iguales en cubetas distintas).

Un objeto Coche crea en su constructor un objeto Motor y este no puede existir sin el coche. La relación entre Coche y Motor se llama...

- a) Herencia
- b) Composición
- c) Polimorfismo
- d) Agregación

RESPUESTA / SOLUCIÓN

b) Composición — Relación fuerte «tiene-un» (has-a) en la que el ciclo de vida del Motor está ligado al del Coche. En la agregación el objeto se crea fuera y se pasa al contenedor.

¿Qué imprime este código?

```
StringBuilder sb = new StringBuilder("abc");
sb.append("def").insert(3, "-");
System.out.println(sb);
```

- a) abc-def
- b) abcdef-
- c) -abcdef
- d) abc-def-

RESPUESTA / SOLUCIÓN

a) abc-def — append("def") deja «abcdef» y insert(3, "-") coloca el guion en el índice 3: «abc-def».

Parte 2: Análisis y depuración

3 puntos — 1,5 ca

El siguiente programa tiene 2 errores. Encuéntralos, explica qué ocurre en cada uno y escribe la versión corregida. (1,5 ptos)

```
public class WrapperBug {
    public static void main(String[] args) {
        Integer n = null;
        int x = n;                // error 1
        Integer a = 500;
        Integer b = 500;
        System.out.println(a == b); // error 2
    }
}
```

RESPUESTA / SOLUCIÓN

1. `int x = n;` con `n = null` lanza `NullPointerException`: el unboxing de un `Integer` `null` no está permitido. Solución: comprobar `n != null` antes o no asignar `null`.
2. `a == b` compara referencias y no valores: con 500 (fuera de la caché) imprime `false`. Solución: usar `a.equals(b)` o `a.intValue() == b.intValue()`.

Versión corregida:

```
Integer a = 500;
Integer b = 500;
System.out.println(a.equals(b)); // true
Integer n = 10;
int x = n;                // unboxing válido: n no es null
System.out.println(x);
```

Realiza la traza del siguiente código e indica qué se imprime en cada línea, explicando la diferencia entre `String` y `StringBuilder`. (1,5 ptos)

```
StringBuilder sb = new StringBuilder("Java");
sb.reverse();
sb.append(" 8");
System.out.println(sb);

String s = "Java";
String s2 = s.toUpperCase();
System.out.println(s);
System.out.println(s2);
```

RESPUESTA / SOLUCIÓN

`sb.reverse()` invierte el contenido del MISMO objeto: «avaJ»; `sb.append(" 8")` añade al final: «avaJ 8». Se imprime: `avaJ 8`.

`String` es inmutable: `toUpperCase()` NO modifica `s`, sino que devuelve un objeto nuevo. Se imprime «Java» (`s` intacto) y «JAVA» (`s2`).

Conclusión: `StringBuilder` modifica el objeto en el sitio (mutable); `String` crea un objeto nuevo en cada operación.

Parte 3: Ejercicios de programación

8 puntos — 2 cada

Crea la clase **CuentaBancaria** con un enum interno **TipoCuenta** { **AHORRO**, **CORRIENTE** }, los atributos privados **numero** (String), **saldo** (double) y **tipo**, y los métodos **getNumero()**, **getSaldo()**, **getTipo()** e **ingresar(double)**. Sobrescribe **equals()** y **hashCode()** para que dos cuentas sean iguales si tienen el mismo número. Escribe un **main** que cree dos cuentas con el mismo número y distintos tipos y compruebe que **equals()** devuelve true. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.Objects;

public class CuentaBancaria {
    enum TipoCuenta { AHORRO, CORRIENTE }

    private String numero;
    private double saldo;
    private TipoCuenta tipo;

    public CuentaBancaria(String numero, TipoCuenta tipo) {
        this.numero = numero;
        this.tipo = tipo;
        this.saldo = 0;
    }

    public String getNumero() { return numero; }
    public double getSaldo() { return saldo; }
    public TipoCuenta getTipo() { return tipo; }

    public void ingresar(double cantidad) { saldo += cantidad; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof CuentaBancaria)) return false;
        CuentaBancaria c = (CuentaBancaria) o;
        return numero.equals(c.numero);
    }

    @Override
    public int hashCode() {
        return Objects.hash(numero);
    }

    public static void main(String[] args) {
        CuentaBancaria c1 = new CuentaBancaria("ES1234", TipoCuenta.AHORRO);
        CuentaBancaria c2 = new CuentaBancaria("ES1234", TipoCuenta.CORRIENTE);
        c1.ingresar(100);
        System.out.println(c1.equals(c2));           // true: mismo número
        System.out.println(c1.getSaldo());           // 100.0
        System.out.println(TipoCuenta.AHORRO.ordinal()); // 0
    }
}
```

```
}  
}
```

`equals()` ignora saldo y tipo: la identidad de la cuenta es su número. `hashCode()` usa el mismo campo para mantener el contrato.

Escribe un programa que lea una frase por teclado y la imprima con las palabras en orden inverso usando `StringBuilder`. Ejemplo: «hola mundo java» → «java mundo hola».
(2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.Scanner;  
  
public class InvertirFrase {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Frase: ");  
        String frase = sc.nextLine();  
        String[] palabras = frase.split(" ");  
        StringBuilder sb = new StringBuilder();  
        for (int i = palabras.length - 1; i >= 0; i--) {  
            sb.append(palabras[i]);  
            if (i > 0) sb.append(" ");  
        }  
        System.out.println("Invertida: " + sb);  
    }  
}
```

Se recorre el array de palabras de atrás hacia adelante y se concatenan con `StringBuilder`, que es eficiente porque no crea un `String` nuevo en cada `append`.

Implementa mediante composición las clases `Motor` (atributo `caballos` y método `arrancar()` que imprime «Motor de X CV en marcha») y `Coche` (marca y un `Motor` creado en su constructor). El método `arrancar()` del `Coche` debe delegar en el motor. Escribe un `main` que cree un `Coche` y lo arranque, y justifica por qué es composición y no herencia. (2 pts)

RESPUESTA / SOLUCIÓN

```
public class CocheMain {  
    static class Motor {  
        private int caballos;  
        Motor(int caballos) { this.caballos = caballos; }  
        void arrancar() {  
            System.out.println("Motor de " + caballos + " CV en marcha");  
        }  
    }  
    static class Coche {  
        private String marca;  
        private Motor motor;           // composición: Coche TIENE-UN Motor  
        Coche(String marca, int caballos) {  
            this.marca = marca;  
            this.motor = new Motor(caballos); // el Motor nace con el Coche  
        }  
    }  
}
```

```

    }
    void arrancar() { motor.arrancar(); } // delega en el motor
}
public static void main(String[] args) {
    Coche c = new Coche("Seat", 110);
    c.arrancar();
}
}

```

Es composición porque Coche no «es un» Motor (no hereda), sino que lo contiene; además el Motor se crea dentro del constructor del Coche, de modo que su ciclo de vida está ligado al del Coche. La salida es: Motor de 110 CV en marcha.

Escribe un programa que lea 5 números por teclado como String, los convierta con Integer.parseInt, los sume con autoboxing/unboxing (variable Integer total) y muestre el resultado. Debe capturar NumberFormatException si el texto no es un número y pedir el valor de nuevo. (2 ptos)

RESPUESTA / SOLUCIÓN

```

import java.util.Scanner;

public class SumaNumeros {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Integer total = 0;           // autoboxing
        for (int i = 0; i < 5; i++) {
            System.out.print("Número " + (i + 1) + ": ");
            String texto = sc.nextLine();
            try {
                int n = Integer.parseInt(texto); // String -> int
                total = total + n;              // unboxing + autoboxing
            } catch (NumberFormatException e) {
                System.out.println("'" + texto + "' no es un número válido");
                i--;                          // repetir la lectura
            }
        }
        System.out.println("Suma: " + total);
    }
}

```

Integer.parseInt convierte el texto; si falla lanza NumberFormatException (subclase de IllegalArgumentException, no comprobada), que se captura para repetir la lectura.

Parte 4: Pregunta teórica

1 punto

Explica las diferencias entre String, StringBuilder y StringBuffer (inmutabilidad, eficiencia y sincronización) y en qué situaciones usarías cada uno.

RESPUESTA / SOLUCIÓN

String es inmutable: cualquier operación (concatenación, toUpperCase, etc.) crea un objeto nuevo. Es eficiente para valores que no cambian y para usarlo como clave en mapas.

StringBuilder es mutable y no sincronizado: modifica el mismo objeto, por lo que es la opción recomendada para concatenar muchas cadenas en un bucle (evita crear decenas de objetos intermedios).

StringBuffer es igual que StringBuilder pero con métodos sincronizados (thread-safe), lo que lo hace más lento; solo tiene sentido en programas multihilo donde varios hilos comparten el objeto.

Regla práctica: String para texto fijo, StringBuilder para construir cadenas dinámicamente en un solo hilo, StringBuffer solo si hay concurrencia.

Plantilla de respuestas

(Páginas en blanco)